Pythonによるデータ可視化

初めての人向け・**90**分×**8**コマ

長崎大学 病院医療情報部 薗田 光太郎 (Kotaro Sonoda)

目次

はじめに 5
この講座で目指すこと . . . 5
全8コマの構成 . . . 5
準備 . . . 5

I 1日目 11/14 6

1 第1コマ　データ可視化（導入編） 7
1.1 この講座で目指すこと . . . 7
1.2 次の2日間の流れ . . . 7
1.3 例：平均が同じでも、姿はまったく違う . . . 7
1.4 グラフが力を発揮する場面 . . . 8
1.5 逆に、グラフが向かない場面 . . . 8
1.6 演習：図を見て、姿を言葉にする（ペアで40分） . . . 8
1.7 まとめ . . . 8

2 第2コマ　Python入門 9
2.1 Thonny の準備 . . . 9
2.2 Thonny の画面 . . . 10
2.3 基本の使い方：4つのステップ . . . 10
2.4 データのダウンロード（実習の前に） . . . 11
2.5 Python は「計算と指示」ができる電卓 . . . 11
2.6 変数：名前のついた箱 . . . 11
2.7 リストと辞書 . . . 12
2.8 繰り返しと条件分岐 . . . 12
2.9 関数とライブラリ . . . 12
2.10 演習 . . . 12
2.11 まとめ . . . 12

3 第3コマ　データ集め、データ（変数）の尺度 13
3.1 データ集めの主な方法 . . . 13
3.2 「度量衡」という言葉について . . . 13
3.3 変数と尺度：データには「ものさし」がある . . . 13
3.4 尺度で「できること」が違う . . . 13
3.4.1 数字でも、名義尺度として扱うもの . . . 14
3.5 水準と選択肢 . . . 14
3.6 標本という考え方 . . . 14
3.7 尺度とグラフの相性（目安） . . . 14
3.8 実習：データを読み込んで、列の尺度を考える . . . 15
3.9 演習 . . . 15
3.10 まとめ . . . 16

4 第4コマ　表のいろいろ・表作成 17
4.1 表の4段階 . . . 17

4.2 段階1：収集（一覧表） . . . 17
4.3 段階2：要約（代表値・統計値の表） . . . 18
4.4 段階3：関係（クロス集計表） . . . 18
4.5 段階4：推測と混同行列 . . . 19
4.6 信号検出理論：刺激のあるなし × 反応のあるなし . . . 19
4.7 表を作るときの約束 . . . 20
4.8 演習 . . . 20
4.9 まとめ（1日目の振り返り） . . . 20

II 2日目 11/15 **21**

5 第5コマ　グラフのいろいろ・伝わるデザイン・グラフ作成 **22**
5.1 グラフは「知りたいこと」で選ぶ . . . 22
5.2 各グラフは「何の、どんな様子」を見せるか . . . 22
5.3 pandas のグラフは「表から直接」 . . . 23
5.4 折れ線・棒・散布図 . . . 23
5.5 ヒストグラム：分布の形を見る . . . 25
5.6 箱ひげ図：分布を要約して比べる . . . 25
5.6.1 四分位数：データを小さい順に「4つに分ける」 . . . 26
5.6.2 箱ひげ図の描き方 . . . 27
5.6.3 外れ値（ハズレ値）：飛び抜けた点の扱い . . . 27
5.7 伝わるデザインの約束 . . . 28
5.8 並べて比べる . . . 30
5.9 演習 . . . 31
5.10 まとめ . . . 31

6 第6コマ　データを眺めて、気づいたことをグラフだけで納得させる（データ分析） **32**
6.1 分析の進め方：眺める → 気づく → 確かめる →納得させる . . . 32
6.2 比べるときは「分布の格差」で見る . . . 32
6.3 実習データ：反応時間の実験（80名） . . . 32
6.4 「グラフだけで納得させる」ための工夫 . . . 33
6.5 演習 . . . 33
6.6 まとめ . . . 34

7 第7コマ　仮説を納得させるグラフを作るためのデータ集め **35**
7.1 仮説とは . . . 35
7.2 納得には「比べる相手」が要る . . . 35
7.3 集め方の設計：チェックリスト . . . 35
7.4 偏りを避ける工夫 . . . 35
7.5 下見の例：仮説「音楽があると反応が遅くなるはず」 . . . 35
7.6 データ集め計画シート . . . 36
7.7 実習 . . . 37
7.8 まとめ . . . 37

8 第8コマ　テスト「データを眺めて、気づいたことをグラフで納得させよ」 **38**
8.1 課題文 . . . 38
8.2 課題スターター . . . 38
8.3 進め方の目安（第6コマの手順） . . . 39
8.4 提出前チェックリスト . . . 39
8.5 振り返り . . . 39

付録 40

用語のミニ辞典 40

困ったときの対処表 41

A Slide Index 42
A.1 1日目 42
A.2 2日目 42

はじめに

Pythonによるデータ可視化 ――初めての人向け・90分×8コマ

- 講師 :薗田 光太郎(長崎大学病院 医療情報部)
- 日程：2026年11月14日(土)・15日(日)

この講座で目指すこと

平均などの要約だけでは分からない「データの姿」を、グラフで納得できる形で知ることが、この講座の芯です。

1. 平均などの数字だけでは、データの姿は分からない
2. 表の読み方・作り方を知ると、グラフが読みやすくなる
3. グラフは「知りたいこと」に合わせて選ぶ
4. 最後は、自分の気づきをグラフだけで納得させられるようになる

全8コマの構成

コマ	テーマ
第1コマ	データ可視化(導入編)
第2コマ	Python入門
第3コマ	データ集め、データ(変数)の尺度
第4コマ	表のいろいろ・表作成
第5コマ	グラフのいろいろ・伝わるデザイン・グラフ作成
第6コマ	データを眺めて、気づいたことをグラフだけで納得させる(データ分析)
第7コマ	仮説を納得させるグラフを作るためのデータ集め
第8コマ	テスト「データを眺めて、気づいたことをグラフで納得させよ」

準備

- 環境は **Thonny**（Python本体と編集画面が一緒になった初心者向け無料ソフト)です。インストール方法は第2コマで説明します
- 実習データは data.zip をダウンロードし、展開して出てくる `data` フォルダを、講義フォルダの中に `code` フォルダと並べて置いてください(詳しくは第2コマ)
- 表とグラフには **pandas**、日本語表示には **matplotlib-fontja** を使います。入れ方は第2コマで案内します

Part I

1日目 11/14

1 第1コマ　データ可視化（導入編）

このコマの進め方：講義（40分）→ 図を見て姿を言葉にする演習（40分）→まとめ（10分）。このコマではまだプログラムは書きません。第2コマからThonny で手を動かします。

1.1 この講座で目指すこと

1. 平均などの数字だけでは、データの姿は分からない
2. 表の読み方・作り方を知ると、グラフが読みやすくなる
3. グラフは「知りたいこと」に合わせて選ぶ
4. 最後は、自分の気づきをグラフだけで納得させられるようになる

1.2 次の2日間の流れ

- 1日目：第1コマ 導入 → 第2コマ Python入門 → 第3コマ データと尺度 →第4コマ 表のいろいろ
- 2日目：第5コマ グラフのいろいろ → 第6コマ データ分析 → 第7コマデータ集めの設計 → 第8コマ テスト

1.3 例：平均が同じでも、姿はまったく違う

3つのクラスは、平均点がどれも約65点です。テストの点数をグラフ（ヒストグラム）にすると、姿はこんなに違います。

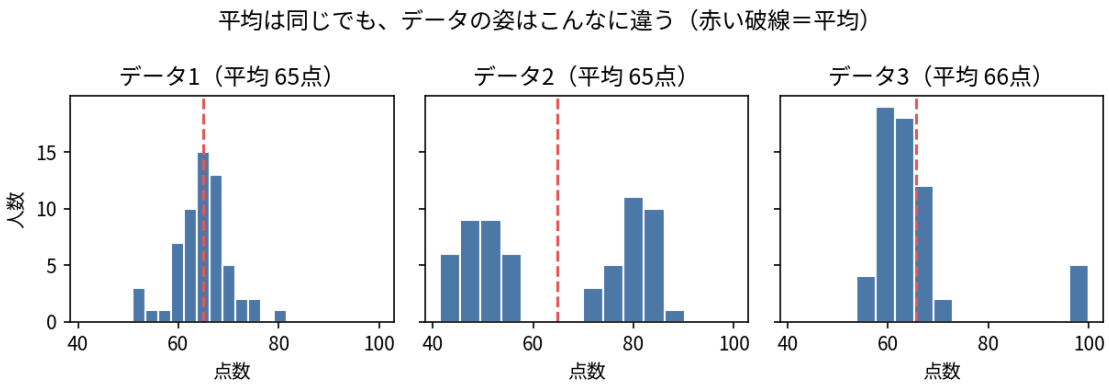


図 1.1: 平均が同じでも姿はまったく違う

平均（要約）だけ見ていては、山の高さ、広がり、偏りといった「姿」は分かりません。グラフで見るからこそ、姿が見えます。

> **ノート**
>
> この図は講師がプログラムで描いたものです。作り方は講座の中で少しずつ身につけます。

1.4 グラフが力を発揮する場面

- たくさんの数を一度に比べたいとき（表は縦に長いと読みにくい）
- 時間の流れや変化を見たいとき
- 「なんとなく多そう」という感覚を、人に納得させたいとき

1.5 逆に、グラフが向かない場面

- 正確な数値そのものを伝えたいとき（例：請求書の金額）は、表や数字のほうがよい
- データが数件しかないときは、数字をそのまま書くほうが伝わる

1.6 演習：図を見て、姿を言葉にする（ペアで40分）

1. 配られた3種類の図（ヒストグラム／箱ひげ図／散布図）をペアで眺める
2. それぞれについて、「どんな姿か」を一文で書く。数字の暗記ではなく、言葉で表す練習
    - 例：「真ん中に山が1つあって、右に長くすそを引いている」
    - 例：「2つのグループは重なっている部分が大きい」
3. ペアで読み合い、相手に伝わる表現だったかを確かめる

1.7 まとめ

- 平均などの要約だけでは、データの姿は分からない
- グラフは、姿を見せ、感覚を「納得」に変える道具
- ただし正確な数値には表の出番もある。使い分けが大事

次回は Thonny を準備して、最初のプログラムを実行します。

2 第2コマ　Python入門

このコマの進め方：Thonny の準備と画面説明（25分）→講義と一緒に実習（55分）→ まとめ（10分）。

2.1 Thonny の準備

- Thonny は、Python 本体と編集画面が一緒になった初心者向けの無料ソフトです（thonny.org）
- Windows へのインストールは、ターミナル（コマンドプロンプト）で次を実行してください

```
winget install AivarAnnamaa.Thonny
```

- うまくいかない場合は、公式サイト thonny.org からダウンロードしてください
- 起動したら、この後に出てくる短いプログラムをそのまま実行してみましょう

2.2 Thonny の画面

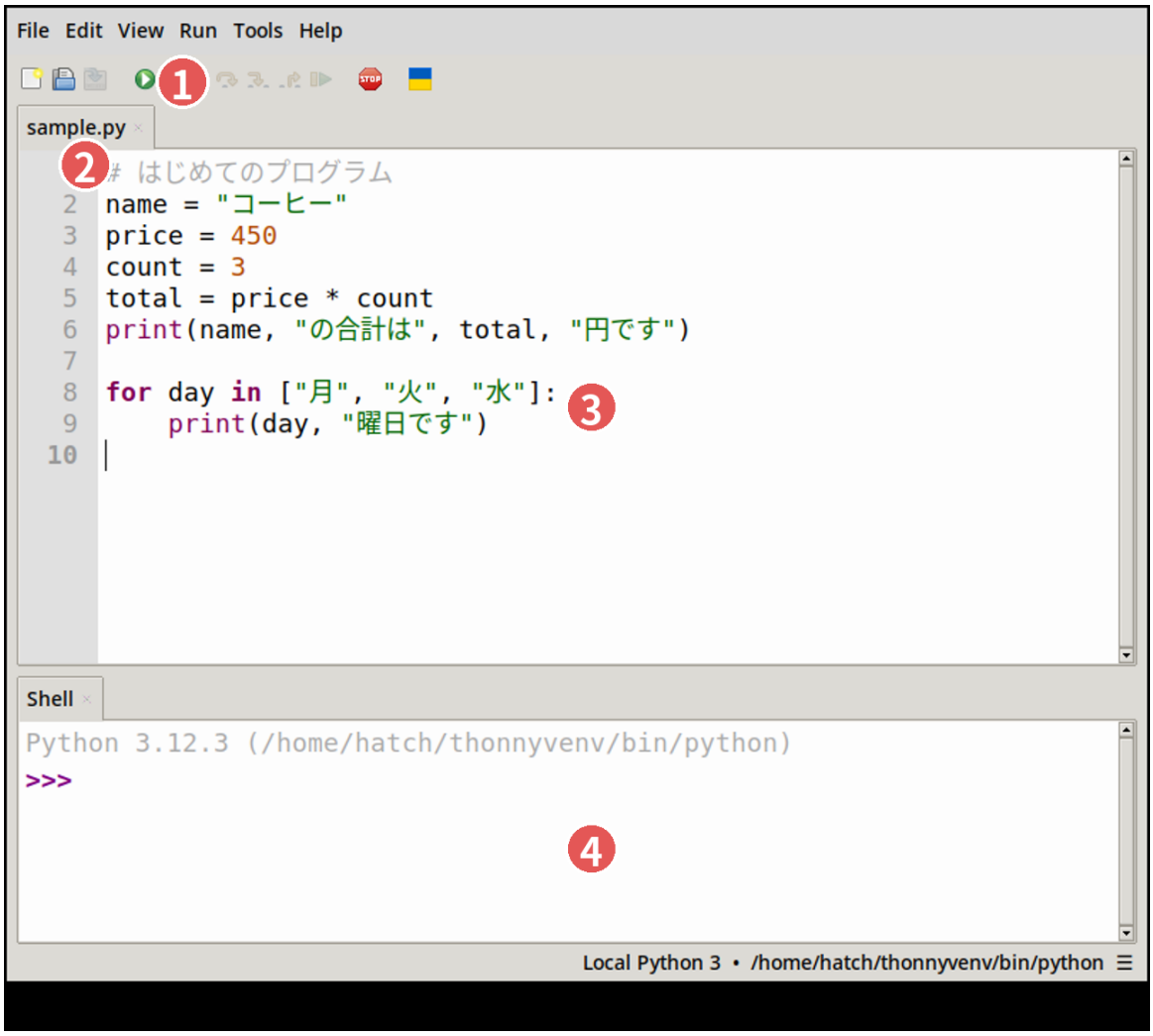


図 2.1: Thonny の画面（番号は下の説明に対応）

1. **実行ボタン**（緑の三角）。押すとプログラムが実行されます（F5キーでも同じ）
2. **ファイルのタブ**。開いているプログラムの名前が出ます
3. **編集エリア**。ここにプログラムを書きます
4. **シェル**。実行結果や、エラーのメッセージが表示されます

（画面は Thonny 5.0 で撮影したものです。表示は設定で少し違うことがあります）

2.3 基本の使い方：4つのステップ

1. **書く** :編集エリアにプログラムを書く
2. **保存する**：Ctrl+S。名前は「session2.py」のように、末尾が .py になるようにします
3. **実行する** :緑の三角ボタン、または F5キー

4. **結果を見る**：文字の結果はシェルに、グラフは新しいウィンドウに表示されます。エラー（赤い文字）が出たら、シェルの一番下の行がヒントです

追加の部品（pandas や matplotlib-fontja）は、メニューの「ツール →パッケージを管理」から入れることができます。第3コマ以降で使います。

2.4 データのダウンロード（実習の前に）

- 実習で使うデータは、あらかじめ固めてある **data.zip をダウンロード**してください
- 配布ページ（data.zip）：https://github.com/helmenov/lecture-dataviz/blob/main/data.zip
- ページ右側のダウンロードボタンで落とし、**展開（解凍）すると data フォルダが出てきます**
- **置き場所**：ドキュメントフォルダの中に講義用のフォルダ（例：dataviz）を作り、その中にcode フォルダと**並べて**置きます

```
ドキュメント/dataviz/      ← 講義フォルダ（自分で作る）
  code/     プログラム（session1.py など）
  data/     ダウンロードしたデータ（csv ファイル）
```

- プログラムは、隣の data フォルダからデータを読みます。**フォルダの並びを崩すと、データが見つからずエラーになります**
- zip で配られている場合は、展開（解凍）してから置いてください。ダウンロードフォルダに置いたままにしないこと
- テスト用のデータ（test_psych.csv）も data フォルダに入っています。第8コマで使います

2.5 Python は「計算と指示」ができる電卓

- プログラムは、上から順に実行されます
- # の後ろはコメント（説明書き）で、実行されません
- まずは `print()` で、計算の答えを表示してみます

```
print("こんにちは")
print(3 + 4)          # 足し算
print(120 * 1.1)      # 消費税込みの価格
```

```
こんにちは
7
132.0
```

（本の中では、このようなプログラムの実行結果もその場で表示されます）

2.6 変数：名前のついた箱

```
price = 120           # 変数 price に 120 を入れる
tax = 1.1
print(price * tax)    # 箱の中身を使って計算する
price = 150           # 入れ直すこともできる
print(price * tax)
```

```
132.0
165.0
```

2.7 リストと辞書

```
scores = [72, 65, 80, 91]   # リスト: 並んだデータ
print(len(scores))          # 個数
print(sum(scores) / len(scores))  # 平均

person = {"名前": "田中", "点数": 72}  # 辞書: 名前付きのデータ
print(person["名前"], person["点数"])
```

```
4
77.0
田中 72
```

2.8 繰り返しと条件分岐

```
for s in [72, 65, 80, 91]:   # リストの中身を1つずつ取り出す
    if s >= 80:
        print(s, "よくできました")
    else:
        print(s, "もう少し")
```

```
72 もう少し
65 もう少し
80 よくできました
91 よくできました
```

2.9 関数とライブラリ

- よく使う処理には名前が付いていて、呼び出すだけです(例：`print`、`len`、`sum`)
- 追加の道具箱が「ライブラリ」。`import` で読み込んで使います
- この講座では、表とグラフの道具箱 **pandas** を中心に使います

2.10 演習

1. 自分の名前と、今日の日付を `print` で表示する
2. リストに家族の年齢を入れて、平均を出す
3. 80点以上なら「合格」、それ以外は「再挑戦」と表示するプログラムに書き換える

2.11 まとめ

- プログラムは上から順に実行される。結果はシェルに出る
- 変数・リスト・繰り返し・条件分岐が分かれば、実習のコードは読める
- 道具箱(ライブラリ)を読み込むと、表とグラフが扱える

3 第3コマ　データ集め、データ（変数）の尺度

このコマの進め方：講義（45分） → 一緒に実習（30分） →演習と振り返り（15分）。

3.1 データ集めの主な方法

- **調査（アンケート）**：聞いて集める。質問の書き方で答えが変わる点に注意
- **観察・記録**：出来事をそのまま記録する。測り方をそろえることが大事
- **実験**：条件を変えて結果を比べる。比べる相手（対照）を用意する
- **既存データ**：公的な統計や、業務でたまったデータを使う。元の集め方を確認する

3.2 「度量衡」という言葉について

このコマで出てくる「度量衡（どりょうこう）」は、昔からの「測るための共通のものさし」のことです。

- 度（ど）＝長さ、量（りょう）＝かさ（体積）、衡（こう）＝重さ
- 今で言えば、メートル・リットル・グラムといった**単位の体系そのもの**
- この後の「比尺度」の正体は、この度量衡です。「誰がどこで測っても同じになる単位で測った数値」とつながります

3.3 変数と尺度：データには「ものさし」がある

- 変数とは、測ったり記録したりする項目のこと（身長、満足度、血液型、気温…）
- 尺度とは、その変数がどんな性質の「ものさし」で測られているか、という区別です
- 尺度が違うと、**やってよい計算や、合うグラフが変わります**

尺度	例	イメージ
名義尺度	血液型、商品名、天気	名札。区別だけ
順序尺度	順位、混雑具合、言葉の満足度（不満～満足）	順番はあるが間隔は不ぞろい
間隔尺度	気温（℃）、西暦、数値化した満足度	間隔は均等。0は「無」ではない
比尺度	身長、金額、個数	度量衡（長さ・重さなどの共通単位）。0＝本当に無い

3.4 尺度で「できること」が違う

尺度	数える・割合	順位	差を言う（較べる）	倍率を言う（比べる）	平均
名義	できる	できない	できない	できない	できない
順序	できる	できる	できない	できない	注意

尺度	数える・割合	順位	差を言う（較べる）	倍率を言う（比べる）	平均
間隔	できる	できる	できる	できない	できる
比	できる	できる	できる	できる	できる

- 差を言う場合（気温）：較べる。5 ℃差と5 ℃差は同じ
- 倍率を言う場合（重さ）：比べる。「2倍重い」が言える
- 間隔尺度では、倍率は言えません（20 ℃は10 ℃の「2倍暑い」ではない）
- 度量衡でない数値データ（満足度など感覚を数値化したもの）も、**間隔尺度として扱います**。満足度は平均も使いますが、「4点は2点の2倍満足」とは言えません

3.4.1 数字でも、名義尺度として扱うもの

- 背番号、郵便番号、商品コードのように、順序の意味（大小・強弱・優劣）がない数値は、**数字符号**として名義尺度扱いです

3.5 水準と選択肢

- **水準**：変数が取りうる値のこと。天気なら「晴れ・くもり・雨」が水準です
- **選択肢**：調査で答えてもらうために用意する水準の並び
- 選択肢の約束：もれがない（「その他」を用意）、重ならない、順序のあるものは順序を保つ
- 文字列の水準でも順序があれば順序尺度（例：順位、混雑具合、言葉の満足度）。数値に割り振り直せば、間隔尺度として扱ってよい

3.6 標本という考え方

- 本当に知りたい全体を**母集団**、実際に集めたデータを**標本（サンプル）**と言います
- 講座のデータも、たいていは標本です（例：来場者全員ではなく、協力してくれた80名）
- 結論はまず、「この標本では、こう見えた」と言える範囲に置くのが安全です
- 標本の偏り（若い人ばかり、協力的で元気な人ばかり等）があると、全体の姿とずれることがあります

3.7 尺度とグラフの相性（目安）

尺度	合うグラフ	例
名義	棒グラフ（人数・割合）	血液型ごとの人数
順序	棒グラフ（人数・割合）	満足度ごとの人数
間隔・比	ヒストグラム／箱ひげ図／折れ線／散布図	身長の分布、気温の変化

読み取りのポイント：**名義・順序は「人数を数えて棒グラフ」。平均の棒グラフにはしません**。間隔・比は分布を見るヒストグラムや箱ひげ図が使えます。

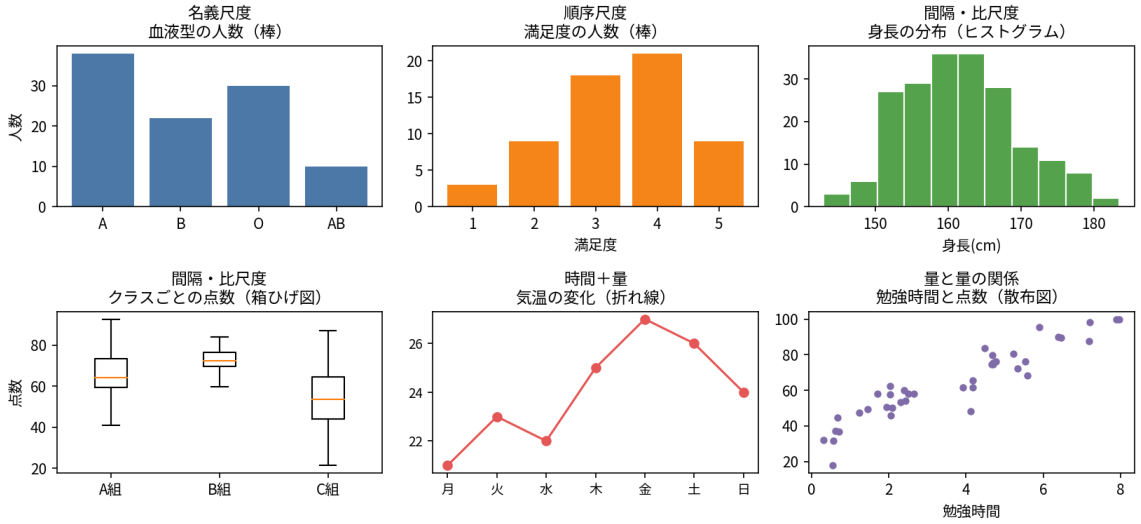


図 3.1: 尺度ごとの例となるグラフ

3.8 実習：データを読み込んで、列の尺度を考える

```
survey = pd.read_csv("data/survey.csv")
print(survey.head())
print(survey["年代"].unique())          # 水準を取り出す
print(survey["満足度"].value_counts())
```

```
    年代   満足度   来店回数
0  20代     5     16
1  10代     5     10
2  40代     1      3
3  30代     3      7
4  10代     2     18
<StringArray>
['20代', '10代', '40代', '30代', '50代', '60代以上']
Length: 6, dtype: str
満足度
5     14
4     14
1     12
3     10
2     10
Name: count, dtype: int64
```

列ごとに「これは何尺度か」を言えるようにしておきましょう。

3.9 演習

1. 身の回りのデータを3つ挙げ、それぞれの尺度を答える
2. 次のデータはどの尺度か：背番号、気温、血液型、満足度（1〜5点）、身長、順位

- 背番号：名義（数字符号）。気温：間隔。満足度（数値）：間隔として扱う。順位：順序

3. アンケートの選択肢を1つ作り、「もれ・重なり・順序」の点検をする

3.10 まとめ

- 尺度は4つ（名義・順序・間隔・比）。比尺度の正体は度量衡
- 尺度によって、できる計算と合うグラフが決まる
- データは標本。結論は「この標本ではこう見えた」から始める

4 第4コマ　表のいろいろ・表作成

このコマの進め方：講義（40分）→ 一緒に実習（35分）→演習と振り返り（15分）。表は「段階」ごとに姿が違います。収集 → 要約 →関係 → 推測、の4段階で整理します。

4.1 表の4段階

段階	表の姿	すること
1：収集	一覧表	集めたデータを、そのまま並べる
2：要約	変数ごとに代表値・統計値に置き換えた表	平均・中央値などの代表値や統計値で置き換える
3：関係	クロス集計表（分割表）	2つの変数の組み合わせで数える
4：推測	混同行列	推測の正誤の数を数える（行と列が同じカテゴリ）

4.2 段階1：収集（一覧表）

```
weather = pd.read_csv("data/weather_week.csv")
print(weather.head(10))
```

```
          日付 曜日  最高気温  最低気温  降水量
0  2026-11-09  月    21    13   0.0
1  2026-11-10  火    23    14   0.0
2  2026-11-11  水    22    15   3.5
3  2026-11-12  木    25    16   0.0
4  2026-11-13  金    27    17   0.0
5  2026-11-14  土    26    18  12.0
6  2026-11-15  日    24    15   0.5
```

- pandas では、表全体を **DataFrame** と呼びます
- まず全体を表示し、行数（データの件数）と列（変数）を確認します

```
print(weather.shape)      # (行数, 列数)
print(weather.dtypes)     # 各列の型
```

```
(7, 5)
日付      str
曜日      str
最高気温    int64
最低気温    int64
降水量    float64
dtype: object
```

4.3 段階2：要約（代表値・統計値の表）

一覧表のままでは姿が見えないので、変数ごとに代表値や統計値に置き換えます。

```
rt = pd.read_csv("data/psych_rt.csv")
print(rt[["反応時間", "正答数"]].describe())
```

```
            反応時間        正答数
count   80.000000  80.000000
mean   378.625000  16.150000
std     98.511115   2.038242
min    243.000000  11.000000
25%    327.750000  15.000000
50%    370.000000  16.000000
75%    400.250000  18.000000
max    837.000000  20.000000
```

- `describe()` は、個数・平均・標準偏差・最小・四分位数・最大を一度に並べてくれます
- ここの四分位数（25%、50%、75%）が、第5コマの箱ひげ図につながります

4.4 段階3：関係（クロス集計表）

```
survey = pd.read_csv("data/survey.csv")
print(pd.crosstab(survey["年代"], survey["満足度"]))
```

```
満足度     1  2  3  4  5
年代
10代     3  3  1  0  8
20代     2  2  3  3  1
30代     2  0  1  4  0
40代     2  2  3  2  2
50代     1  3  1  4  2
60代以上   2  0  1  1  1
```

- 割合で見たいときは `normalize="index"` を付けます（行ごとに100%）

```
ct = pd.crosstab(survey["年代"], survey["満足度"], normalize="index")
print((ct * 100).round(1))
```

```
満足度        1     2     3     4     5
年代
10代     20.0  20.0   6.7   0.0  53.3
20代     18.2  18.2  27.3  27.3   9.1
30代     28.6   0.0  14.3  57.1   0.0
40代     18.2  18.2  27.3  18.2  18.2
50代      9.1  27.3   9.1  36.4  18.2
60代以上   40.0   0.0  20.0  20.0  20.0
```

- グループごとの合計や平均は `groupby` で作ります（この例は「一元表」にもなります）

```
sales = pd.read_csv("data/shop_sales.csv")
sales["金額"] = sales["単価"] * sales["個数"]
by_item = (sales.groupby("商品")[["金額", "個数"]].sum()
           .rename(columns={"金額": "金額合計", "個数": "個数合計"})
           .sort_values("金額合計", ascending=False))
print(by_item)
```

```
        金額合計  個数合計
商品
コーヒー  541800  1204
紅茶     292400   731
ジュース  193200   552
ケーキ    189750   345
```

4.5 段階4：推測と混同行列

- 推測とは、あるデータの値から別のデータの値を当てることです。当てた結果の正しさを数えるのが、この段階の仕事です
- 推測の出来を評価する表が**混同行列**です。クロス集計表に似ていますが、**行と列が同じカテゴリ**（実際× 判断）なのが特徴で、斜めのマスが正解です
- 心理学の**信号検出理論**が、その代表例です（次へ）

4.6 信号検出理論：刺激のあるなし × 反応のあるなし

かすかな音が「聞こえたか」を答える実験のように、刺激があるかないかと、反応（「あった」と答えたか）の組み合わせで、判断の様子を調べる理論です。

実際 ＼ 反応	「あった」	「なかった」
刺激あり（50試行）	38 **ヒット**	12 **見落とし**
刺激なし（50試行）	9 **誤警報**	41 **正棄却**

- この人の的中の様子：刺激ありでは76%当て、刺激なしでは18%「あった」と言ってしまう
- 判断の基準がゆるい人は「ヒットも誤警報も多い」、厳しい人は「見落としが多い」といった具合に、4つのマスのバランスに、その人の判断のクセが表れます
- この表はまさに混同行列です。pandas では `pd.crosstab(刺激の列, 反応の列)` で作れます

```
sdt = pd.read_csv("data/sdt_trials.csv")
cm = pd.crosstab(sdt["刺激"], sdt["反応"])
print(cm)
hit_rate = cm.loc["あり", "あった"] / cm.loc["あり"].sum()
fa_rate = cm.loc["なし", "あった"] / cm.loc["なし"].sum()
print(f"ヒット率: {hit_rate:.0%} / 誤警報率: {fa_rate:.0%}")
```

```
反応  あった  なかった
刺激
あり    38     12
なし     9     41
ヒット率: 76% / 誤警報率: 18%
```

4.7 表を作るときの約束

- タイトルと単位を付ける。「何の表か」が一目で分かること
- 数値の桁をそろえる。小数点以下の桁数は列ごとに統一する
- 出典と件数（n＝何件か）を添える

4.8 演習

1. survey.csv で、年代ごとの満足度クロス集計を「割合」の表にし、一文で読み取る
2. shop_sales.csv で、商品ごとの金額合計の一元表を作る（単価×個数＝金額の列を足してから合計）
3. sdt_trials.csv の混同行列から、ヒット率と誤警報率を求める

4.9 まとめ（1日目の振り返り）

- 表は4段階：収集（一覧表）／要約（代表値の表）／関係（クロス集計）／推測（混同行列）
- pandas では read_csv → head/shape → describe/crosstab/groupby が基本の流れ
- 明日は、この表をグラフに変えていきます

Part II

2日目 11/15

5 第5コマ　グラフのいろいろ・伝わるデザイン・グラフ作成

このコマの進め方：講義（40分）→ 一緒に実習（35分）→演習と振り返り（15分）。グラフは pandas の plot で描きます（裏側ではmatplotlib が動いています）。表示と仕上げだけ matplotlib を使います。日本語表示は matplotlib-fontja です。

5.1 グラフは「知りたいこと」で選ぶ

知りたいこと	合うグラフ
時間とともにどう変わったか	折れ線グラフ
量を比べたい（名義・順序）	棒グラフ
2つの量に関係があるか	散布図
ばらつき・偏りを見たい	ヒストグラム
複数のグループの分布を比べたい	箱ひげ図
全体の中の割合	円グラフ／100%積み上げ棒（注意あり）

第3コマの尺度とセットで考えます。「格好いいから」ではなく「質問に答える形」を選ぶのが原則です。

5.2 各グラフは「何の、どんな様子」を見せるか

グラフ	何の、どんな様子を見せたいときに使うか
棒グラフ	カテゴリごとの**量の大きさ**。「どのカテゴリが、どれくらい大きいか」の様子
ヒストグラム	1つの量の**分布の形**。「どこに集まり、どう偏り、山はいくつか、飛び抜けた値はないか」の様子
箱ひげ図	複数のグループの**分布の要約**。「中心はどこか、広がりはどうか、外れ値はあるか」をグループごとに比べる様子
折れ線グラフ	時間とともに量が**どう変化したか**。「増えた・減った・横ばいか、ピークはいつか」の様子
散布図	2つの量の**関係**。「一方が大きいとき、もう一方も大きいか（右上がり／右下がり／無関係）、点はどう集まり、外れ値はないか」の様子

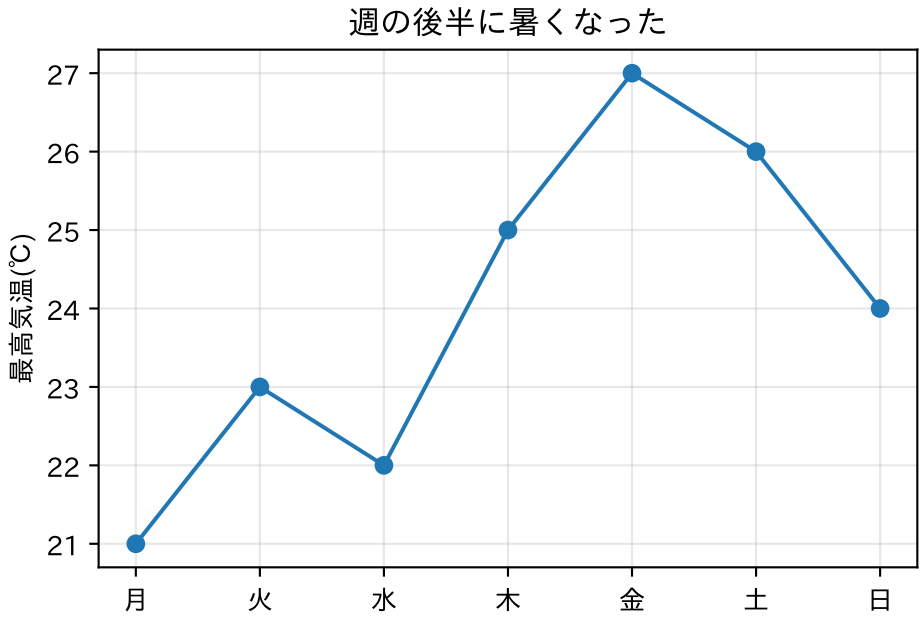
描く前に「何の、どんな様子を見せたいのか」を一言で言えることが、グラフ選びの出発点です。

5.3 pandas のグラフは「表から直接」

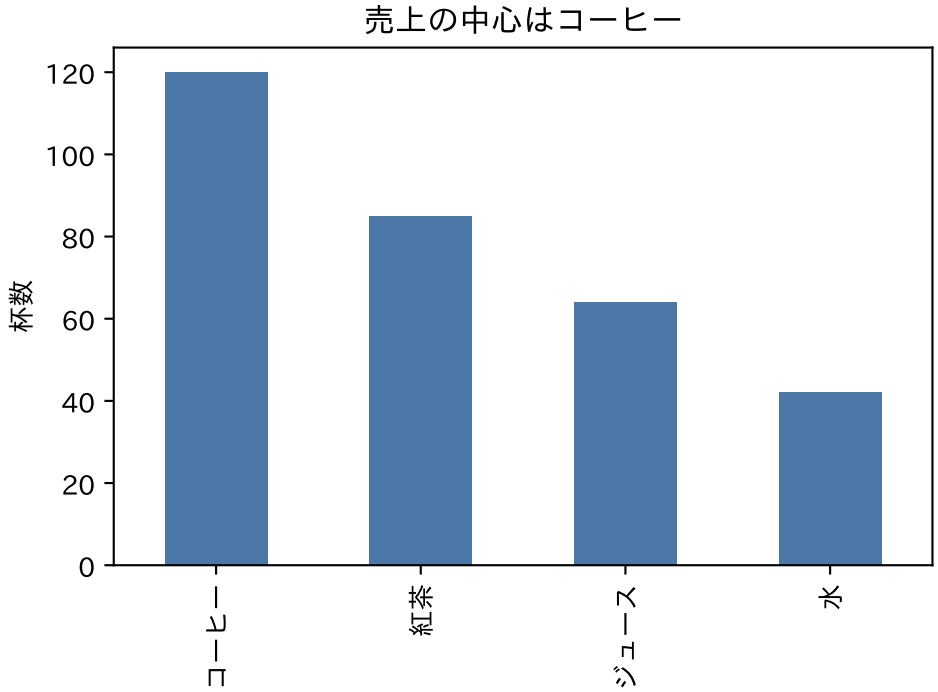
表（DataFrame）の `.plot` に種類（line、bar、scatter など）を指定するだけです。仕上げは matplotlib が担います。

5.4 折れ線・棒・散布図

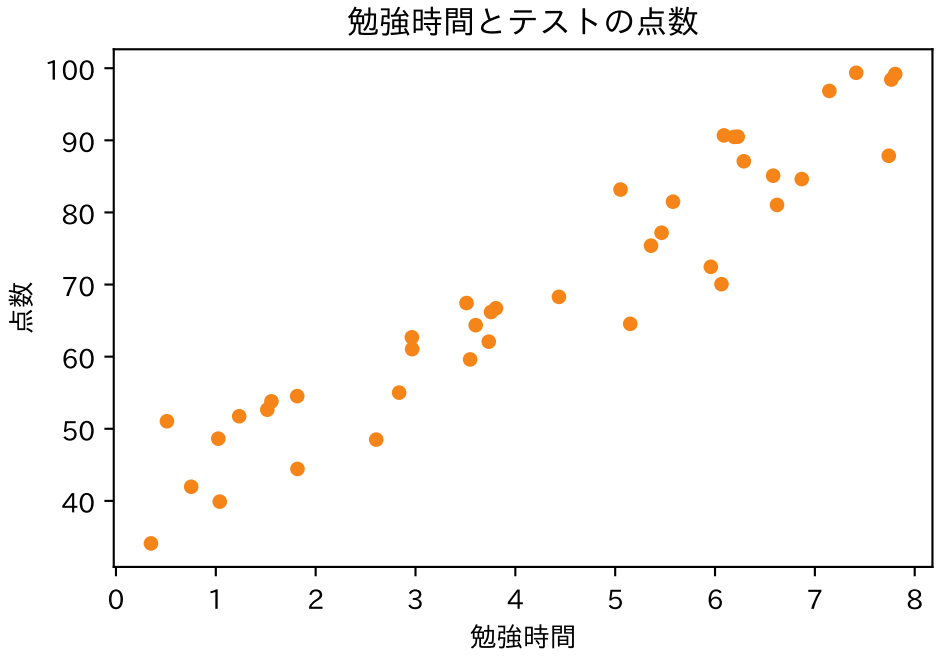
```
temp.plot.line(marker="o", title="週の後半に暑くなった", ylabel="最高気温（℃）")
plt.grid(alpha=0.3)
plt.show()
```



```
sales.plot.bar(color="#4C78A8", title="売上の中心はコーヒー", ylabel="杯数")
plt.show()
```



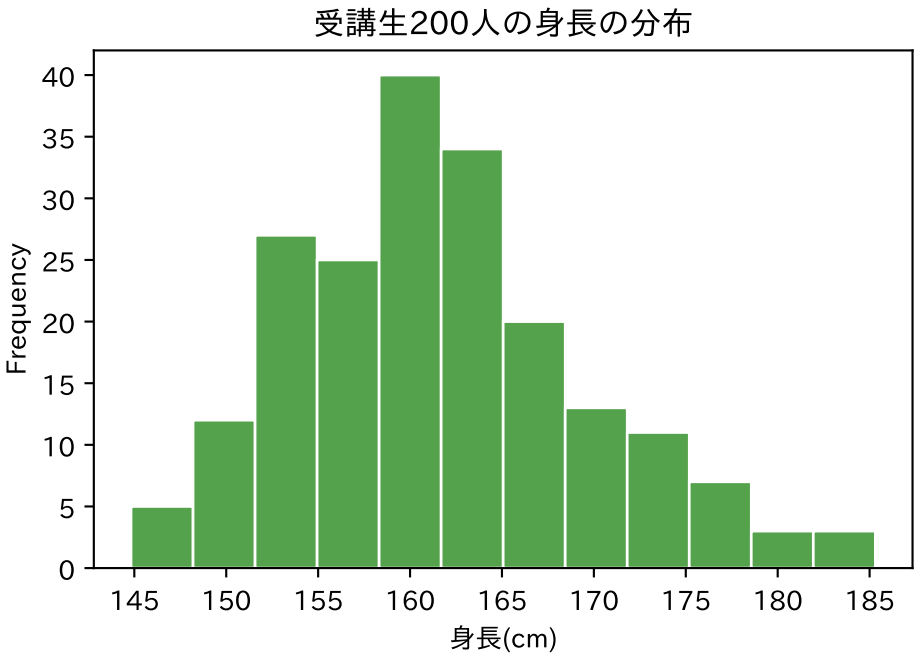
```
study.plot.scatter(x="勉強時間", y="点数", color="#F58518",
                   title="勉強時間とテストの点数")
plt.show()
```



> **ノート**
>
> 円グラフは角度が読みにくいので注意が必要です。3Dグラフは、奥行きで値が読みにくいのでこの講座では扱いません。

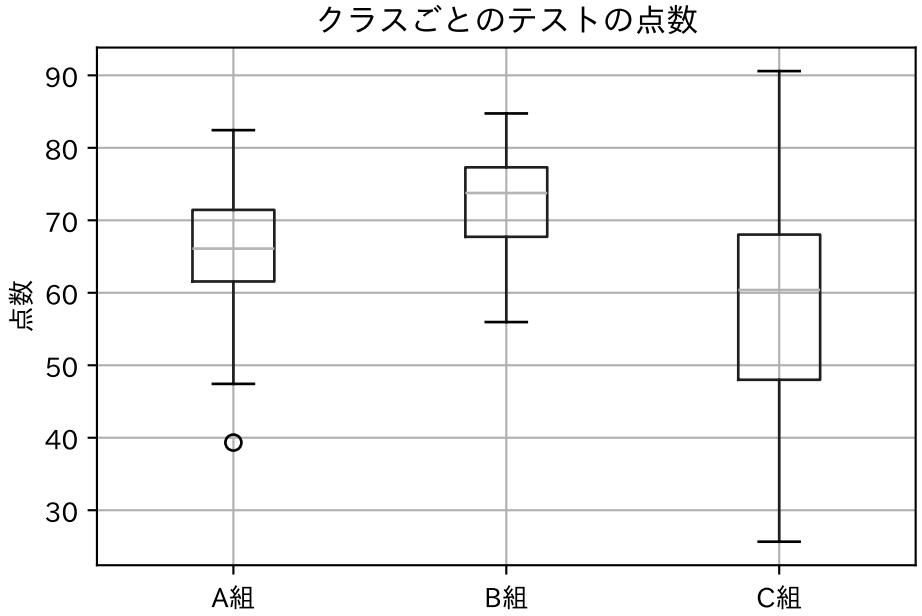
5.5 ヒストグラム：分布の形を見る

```
heights.plot.hist(bins=12, color="#54A24B", edgecolor="white",
                  title="受講生200人の身長の分布", xlabel="身長(cm)")
plt.show()
```



5.6 箱ひげ図：分布を要約して比べる

```
scores.boxplot()
plt.title("クラスごとのテストの点数")
plt.ylabel("点数")
plt.show()
```



5.6.1 四分位数：データを小さい順に「4つに分ける」

箱ひげ図の中身は「四分位数」です。データ全体の真ん中50%が箱に入ります。

- データを小さい順に並べたとき、下から25%の位置が**第1四分位数（Q1）**、50%が**中央値**、75%が**第3四分位数（Q3）**
- 箱の幅 Q3 − Q1 を**四分位範囲（IQR）**と呼びます。真ん中50%の広がりです
- `describe()` の25%・50%・75%（第4コマ）に出ていた、あの行がそのまま箱の上下と中央の線になります
- なお、位置の決め方には流儀があり、手計算とソフトで少しずれることがあります

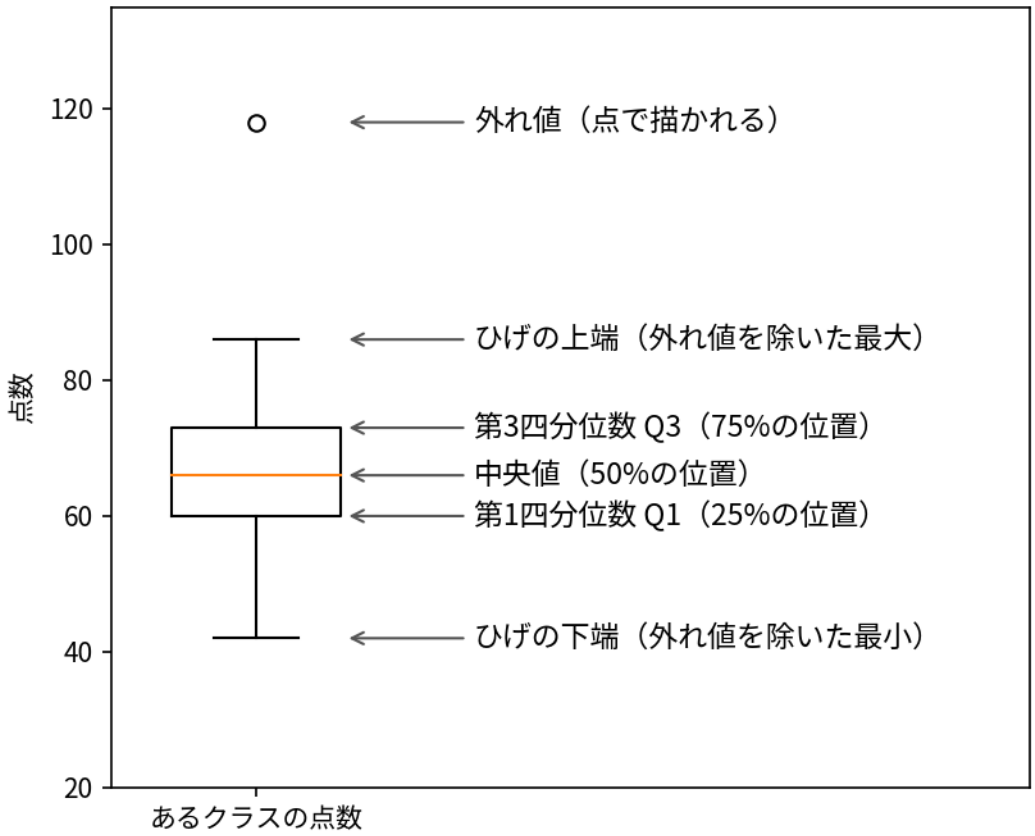


図 5.1: 箱ひげ図の解剖図

5.6.2 箱ひげ図の描き方

1. データを小さい順に並べ、四分位数（Q1・中央値・Q3）を求める
2. Q1 から Q3 までを箱で囲み、中央値に線を引く（太線）
3. 「ひげ」は、箱から 1.5×IQR 以内にある最も端のデータまで伸ばす
4. それより外の点は、1つずつ点で描く（外れ値）

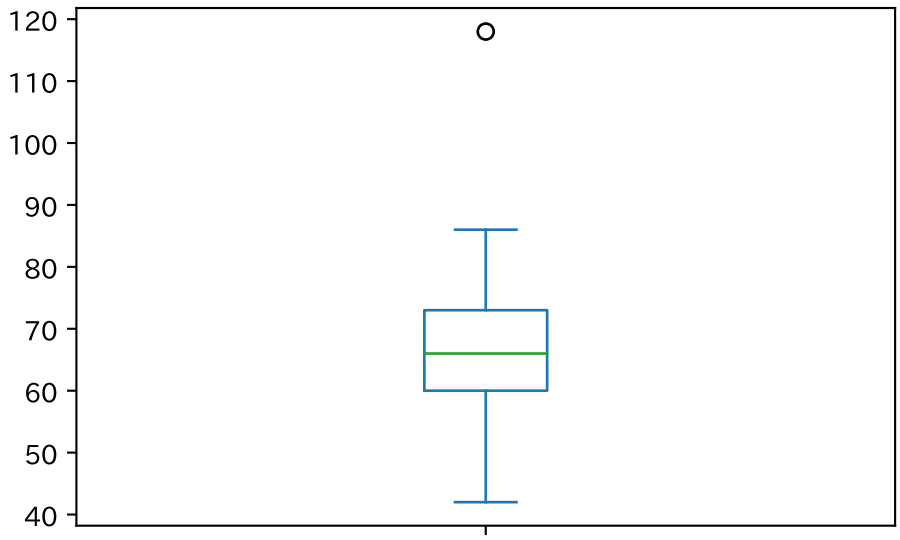
5.6.3 外れ値（ハズレ値）：飛び抜けた点の扱い

- 箱ひげ図で、ひげの外に点で描かれる値が外れ値です
- 目安は数式でも定義されます：Q1 − 1.5×IQR より小さい、または Q3 +1.5×IQR より大きい
- 外れ値は「間違い」と決めつけず、まず理由を調べます（入力ミスか、本当は特別な事情か）
- 消すのはその後。理由なく消すと、分析の結論がゆがみます

```
one_class = pd.Series([42, 48, 52, 55, 57, 58, 60, 61, 62, 63, 64, 65, 66,
                       67, 68, 69, 70, 71, 73, 75, 77, 79, 82, 86, 118])
q1 = one_class.quantile(0.25)
med = one_class.median()
q3 = one_class.quantile(0.75)
```

```
iqr = q3 - q1
print(f"Q1={q1}, 中央値={med}, Q3={q3}, IQR={iqr}")
lo, hi = q1 - 1.5 * iqr, q3 + 1.5 * iqr
print("外れ値の候補:", one_class[(one_class < lo) | (one_class > hi)].tolist())
plt.figure()
one_class.plot.box()
plt.show()
```

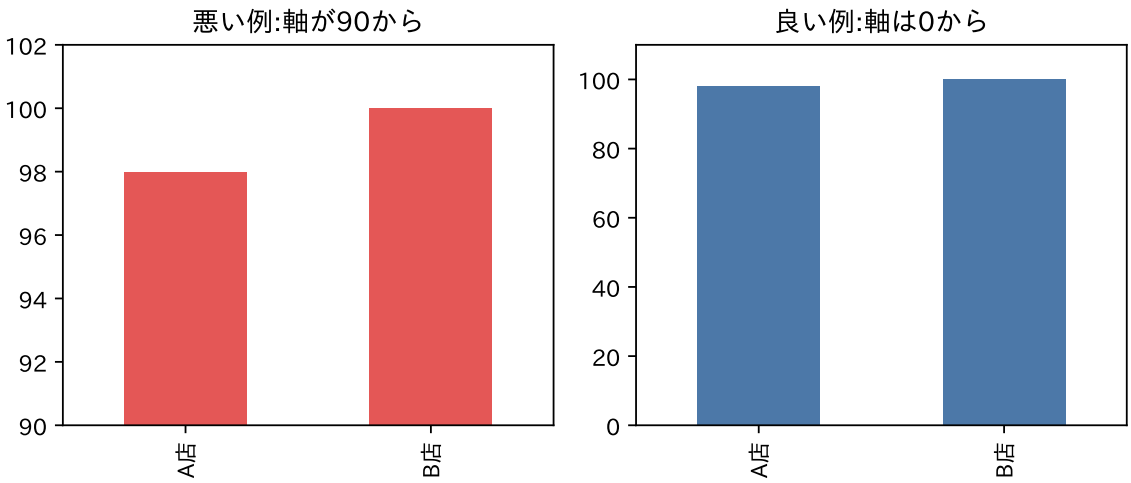
```
Q1=60.0, 中央値=66.0, Q3=73.0, IQR=13.0
外れ値の候補: [118]
```



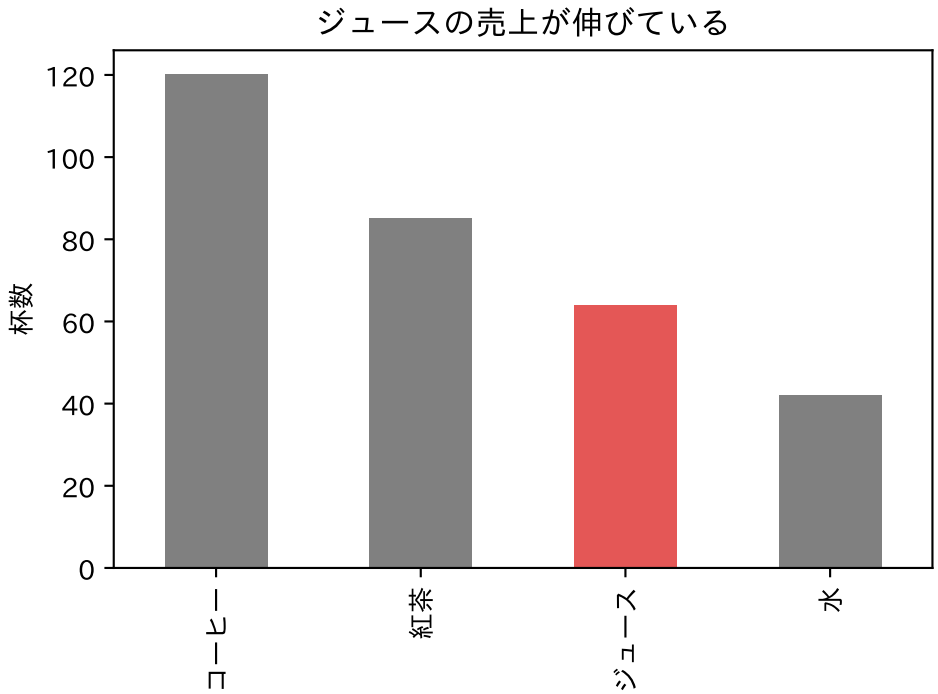
5.7 伝わるデザインの約束

- 軸は原則 **0** から（途中から始めると、差が実際より大きく見えます）
- タイトルは「何の図か」＋できれば主張を一言
- 単位と件数を添える
- 色は少なく。強調したい所だけ色を付ける
- 飾り（影、立体、グラデーション）は付けない

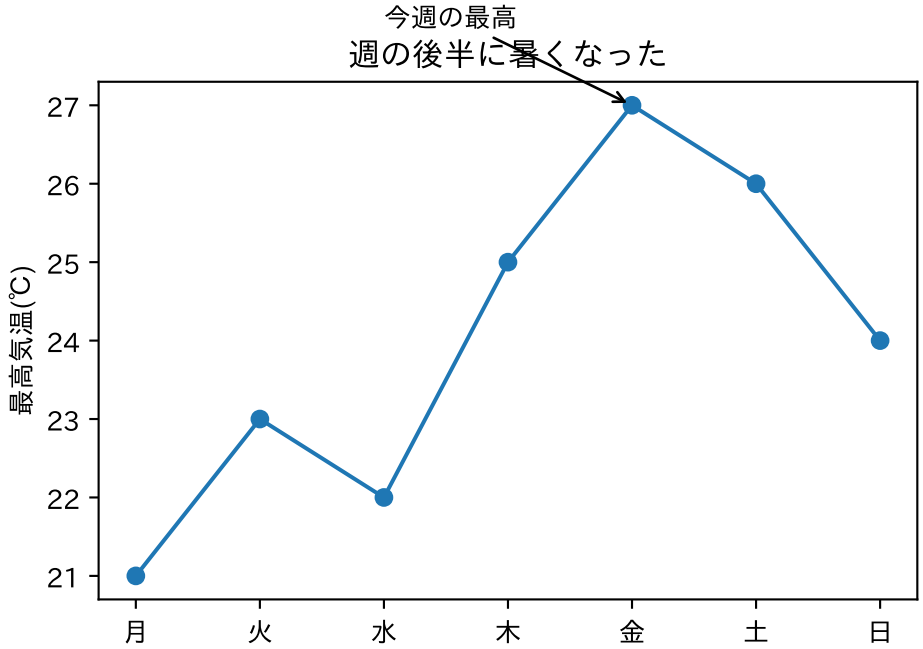
```
vals = pd.Series([98, 100], index=["A店", "B店"])
fig, axes = plt.subplots(1, 2, figsize=(7.2, 3.2))
vals.plot.bar(ax=axes[0], color="#E45756", ylim=(90, 102), title="悪い例:軸が90から")
vals.plot.bar(ax=axes[1], color="#4C78A8", ylim=(0, 110), title="良い例:軸は0から")
plt.tight_layout()
plt.show()
```




```
sales.plot.bar(color=["gray", "gray", "#E45756", "gray"],
               title="ジュースの売上が伸びている", ylabel="杯数")
plt.show()
```

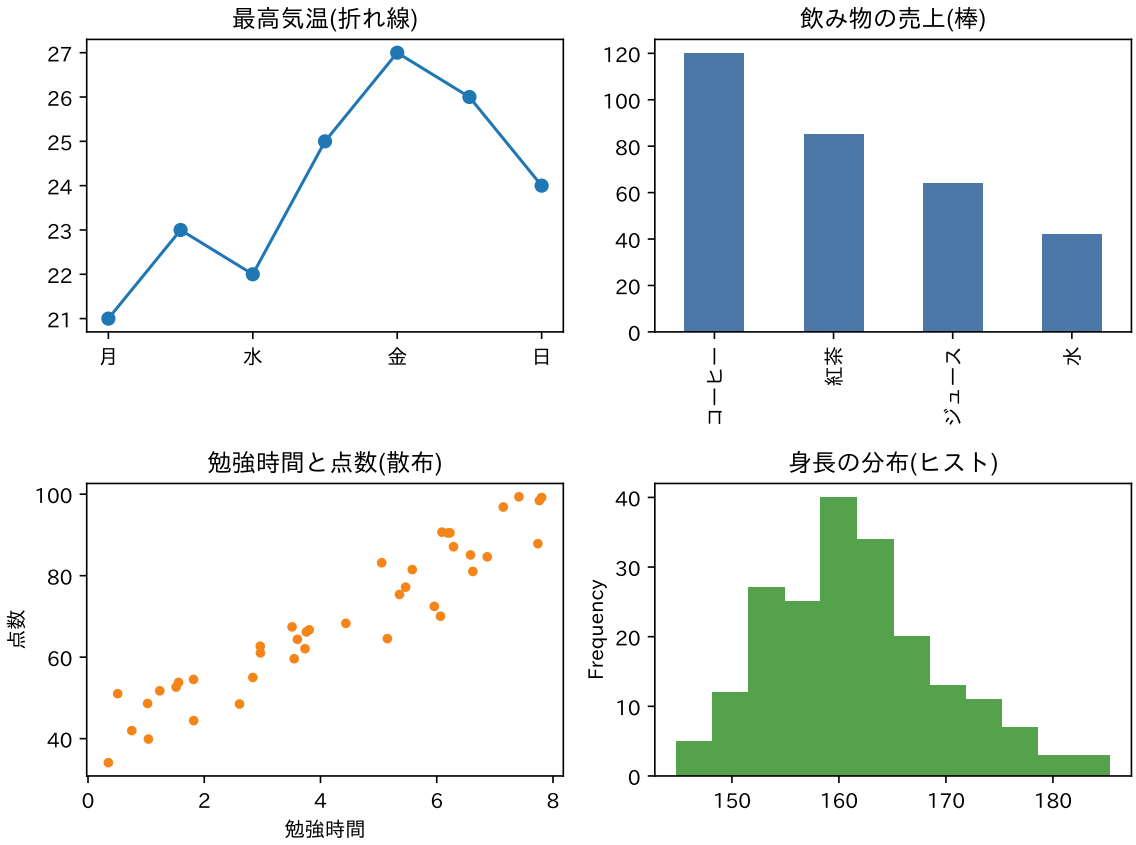



```
ax = temp.plot.line(marker="o", title="週の後半に暑くなった", ylabel="最高気温(℃)")
ax.annotate("今週の最高", xy=(4, 27), xytext=(2, 28),  # 位置は左から数えた番号(月=0…金=4)
            arrowprops=dict(arrowstyle="->"))
plt.show()
```



5.8 並べて比べる

```
fig, axes = plt.subplots(2, 2, figsize=(8, 6))
temp.plot.line(ax=axes[0, 0], marker="o", title="最高気温(折れ線)")
sales.plot.bar(ax=axes[0, 1], color="#4C78A8", title="飲み物の売上(棒)")
study.plot.scatter(ax=axes[1, 0], x="勉強時間", y="点数", s=14,
                   color="#F58518", title="勉強時間と点数(散布)")
heights.plot.hist(ax=axes[1, 1], bins=12, color="#54A24B",
                  title="身長の分布(ヒスト)")
plt.tight_layout()
plt.show()
```



5.9 演習

1. お題(出席率の変化／満足度の偏り／身長と体重の関係)に合う形を描きましょう
2. 軸が90からの棒グラフを、0からに直しましょう
3. タイトルを「結論を書く形」に書き換えましょう

5.10 まとめ

- グラフは「知りたいこと」で選ぶ。各グラフは「何のどんな様子」を見せるためのものか言えること
- 箱ひげ図は四分位数の図。外れ値は、理由を調べてから扱う
- pandas の .plot で描き、matplotlib で仕上げる
- デザインの約束：軸は0から、タイトル＝主張、単位と件数、色は少なく

6 第6コマ　データを眺めて、気づいたことをグラフだけで納得させる（データ分析）

このコマの進め方：講義（20分）→ 講師デモ（15分）→ 一緒に実習（35分）→演習（20分）。

6.1 分析の進め方：眺める → 気づく → 確かめる →納得させる

1. **眺める**：一覧表と要約の表を、しばらく眺める
2. **気づきをメモする**：「音楽があると遅そう」「年代で違いそう」のように、仮の言葉でよい
3. **集計で確かめる**：一元表・クロス集計で、気づきが数字の上でも言えるか見る
4. **グラフに仕上げる**：タイトル＝主張の1枚で、説明が無くても伝わる形にする

6.2 比べるときは「分布の格差」で見る

- 第6コマの比較は、平均の差だけでは決めません。**分布の格差（中心・広がり・重なり）**もとに行います
  - 中心：中央値（箱の線）がどれくらいずれているか
  - 広がり：箱やひげの幅が、グループでどれくらい違うか
  - 重なり：2つの分布がどれくらい重なっているか（重なりが大きいと「違う」と言いにくい）
- 箱ひげ図は、この3つが一度に読めるので比較に向きます

6.3 実習データ：反応時間の実験（80名）

- 列：ID、年代、条件（静か／音楽あり）、反応時間（ミリ秒）、正答数（20問中）（psych_rt.csv）
- 画面に合図が出たらボタンを押す、という課題を80名に実施した記録です
- まず一覧を眺め、条件ごと・年代ごとの要約の表を作ってみましょう

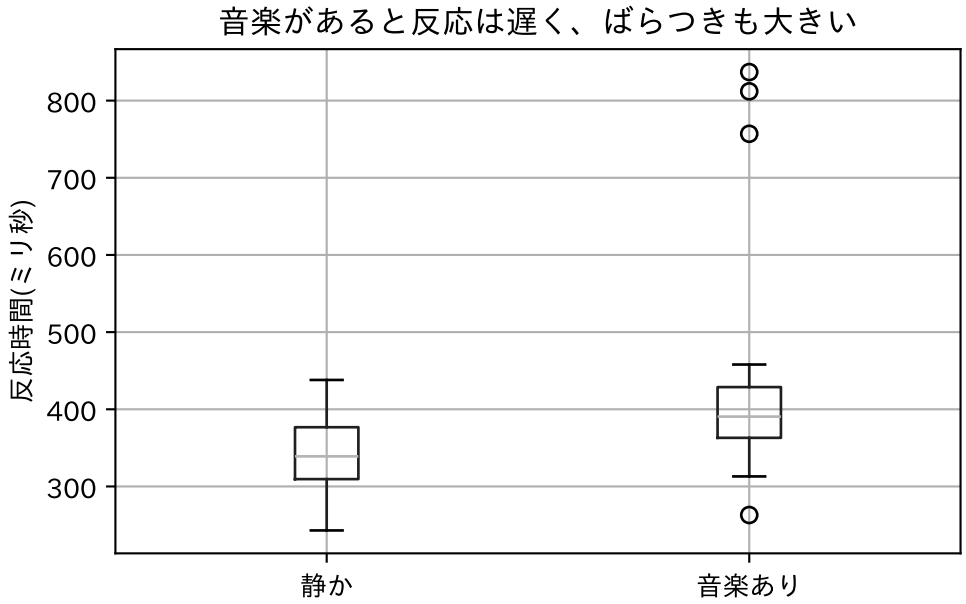
```
rt = pd.read_csv("data/psych_rt.csv")
print(rt.head(10))
print(rt.groupby("条件")["反応時間"].agg(["mean", "median", "std", "count"]))
```

```
   ID   年代     条件   反応時間   正答数
0   1  60代   音楽あり    432    19
1   2  20代   音楽あり    315    15
2   3  20代   音楽あり    353    12
3   4  60代   音楽あり    451    19
4   5  40代   音楽あり    428    11
5   6  20代     静か    350    19
6   7  40代   音楽あり    387    17
7   8  40代   音楽あり    370    14
8   9  40代   音楽あり    363    13
9  10  40代   音楽あり    363    13
          mean  median         std  count
```

```
条件
静か      340.925    339.0    49.819874       40
音楽あり    416.325    390.5   119.416184       40
```

平均と中央値がずれる条件がある → 外れ値の気配があります。

```
rt.boxplot(column="反応時間", by="条件")
plt.title("音楽があると反応は遅く、ばらつきも大きい")
plt.suptitle("")
plt.xlabel("")
plt.ylabel("反応時間(ミリ秒)")
plt.show()
```



6.4 「グラフだけで納得させる」ための工夫

- タイトルを**主張そのもの**にする(「音楽ありのほうが遅い」)
- 比べる相手を同じ図の中に置く
- 強調色と注釈で、見てほしい場所を指す
- 説明文を添えなくても主張が伝わるか、隣の人に見せて確かめる

6.5 演習

1. 反応時間のデータから、自分の「気づき」を **1つ**選ぶ
2. それを、タイトル＝主張のグラフ1枚で表す。説明の文章は書かない。比べる場合は、平均だけでなく分布を並べた図にすること
3. 隣の人と見せ合い、主張が一目で伝わるか確かめ合う

6.6 まとめ

- 分析は「眺める→気づく→確かめる→グラフで納得させる」の順
- 比較は、平均ではなく分布の格差（中心・広がり・重なり）で見る
- グラフだけで納得させるには、タイトル＝主張と比較相手が要る

7 第7コマ　仮説を納得させるグラフを作るためのデータ集め

このコマの進め方：講義（30分）→ 一緒に実習（25分）→ 班の実習（25分）→発表とまとめ（10分）。

7.1 仮説とは

- 仮説とは「もし○○なら、△△になるはず」という、確かめられる形の予想です
- 例：「音楽があると反応が遅くなるはず」「練習を重ねると反応が速くなるはず」
- あいまいな予想（「なんとなく速そう」）は、データで確かめられません
- 納得させるグラフを作るには、**仮説に合ったデータ**が要ります。第6コマのように手元のデータを眺める段階とは、ここが違います

7.2 納得には「比べる相手」が要る

- 「音楽ありの反応時間は390ミリ秒でした」だけでは、遅いのか分かりません
- 静かな条件（中央値339ミリ秒）と並べて初めて、「音楽があると遅い」と納得できます
- 比べ方は仮説で決まります：静か vs 音楽あり／実施前 vs 実施後／若い年代vs 高い年代
- 計画の時点で「何と何を比べるか」を決めておくことが、集め方の設計です

7.3 集め方の設計：チェックリスト

- 必要な変数はそろっているか（尺度も意識する。第3コマ）
- 比べる2群の**条件は、比べたい点以外そろっているか**
- 件数は足りるか。少なすぎると、たまたまの偏りに振り回されます
- 測り方・記録の仕方は全員で同じか
- 第3コマの「標本」の話：集めた相手は、知りたい全体を代表しているか

7.4 偏りを避ける工夫

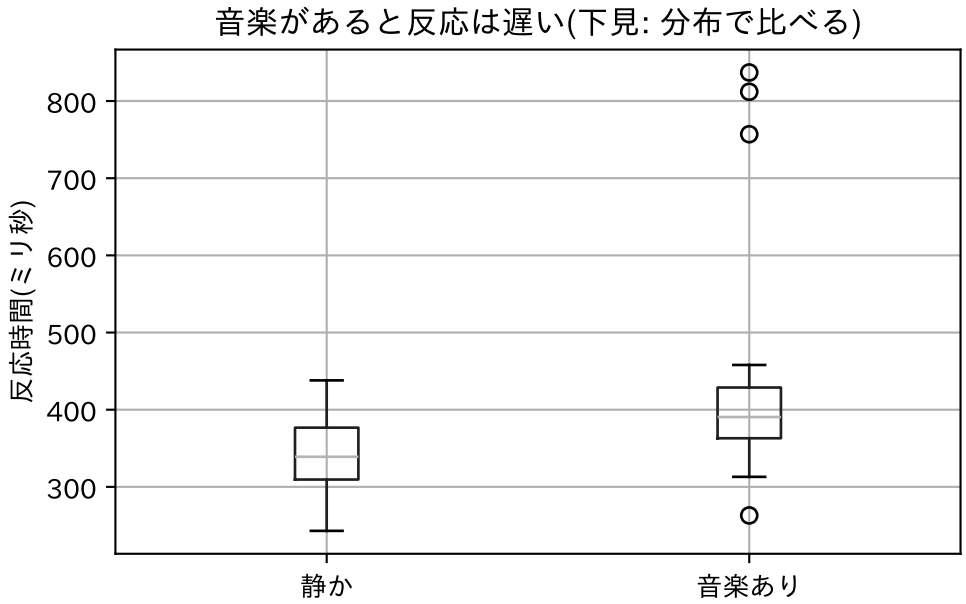
- **集める相手を偏らせない**：都合のよい人だけ集めると、姿がゆがみます
- **比べる群の割り付けを偏らせない**：元気な人ばかり片方の群に入れない
- **測る条件をそろえる**：時間帯や場所が群ごとに違うと、何の差か分からなくなります
- 記録の欠け（欠けた値）は、無いものとして扱わず、件数を数えておきます

7.5 下見の例：仮説「音楽があると反応が遅くなるはず」

```
rt = pd.read_csv("data/psych_rt.csv")
cmp_table = (rt.groupby("条件")["反応時間"].agg(["mean", "median", "count"])
             .rename(columns={"mean": "平均", "median": "中央値", "count": "人数"}))
print(cmp_table)
```

```
              平均      中央値   人数
条件
静か      340.925   339.0   40
音楽あり   416.325   390.5   40
```

```
rt.boxplot(column="反応時間", by="条件")
plt.title("音楽があると反応は遅い(下見: 分布で比べる)")
plt.suptitle("")
plt.xlabel("")
plt.ylabel("反応時間(ミリ秒)")
plt.show()
```



7.6 データ集め計画シート

項目	書き方
仮説	「もし○○なら、△△になるはず」
比べる相手	何と何を比べるか
必要な変数	（尺度も書く）
集め方	調査／観察／実験／既存データ
相手と数	誰から・何件
期間	いつからいつまで
偏りへの対策	割り付け・条件そろえなど

記入例：「朝の課題は、夕方の課題より反応が速いはず」／比べる相手＝夕方に実施したときの反応時間／変数＝時間帯（名義）・反応時間（比）・年代（名義）／実験で集める／各時間帯30名ずつ、年代の内訳がそろうようにする／同じ人が両方やると練習効果が出るので、人は分ける。

7.7 実習

1. 班ごとに仮説を1つ立てる
2. 計画シートを完成させる
3. 既にあるデータ（反応時間の実験データなど）で「下見」ができる場合は、比べる表だけ作ってみる
4. 発表は「仮説」と「比べる相手」の2点を中心に

7.8 まとめ

- 仮説が先、集め方は後。比べる相手は計画の時点で決める
- 偏りは、あとから直せません。設計で避ける
- 次のコマはテストです。各自、データから気づきをグラフで納得させます

8 第8コマ　テスト「データを眺めて、気づいたことをグラフで納得させよ」

このコマの進め方：課題の説明（10分） → テスト実施（70分） →振り返り（10分）。2日間の総合テストです。第1〜7コマの道具を、自分の力で使います。

8.1 課題文

配布されたデータ（test_psych.csv：ある注意課題の実験に参加した120名の記録）を眺めて、気づきを **1** つ挙げ、それを**グラフだけで納得させる**形で示しなさい。

- 列：ID、年代、性別、条件（単独／ペア）、時間帯（午前／午後）、反応時間（ミリ秒）、正答数（20問中）
- 提出物：①気づき（1文） ②その気づきを納得させるグラフ（タイトル＝主張） ③使ったプログラム
- 比べる場合は、平均だけでなく**分布の格差（中心・広がり・重なり）**が見える形にすること
- 制限時間70分

8.2 課題スターター

```
df = pd.read_csv("data/test_psych.csv")
print(df.head())
print(df.describe(include="all"))
# ここから先は自分で書く
```

```
   ID   年代  性別  条件 時間帯    反応時間   正答数
0   1  20代  女性  ペア  午前  295.0    17
1   2  20代  女性  ペア  午後  311.0    13
2   3  20代  女性  単独  午前  339.0    16
3   4  40代  男性  単独  午後  343.0    18
4   5  40代  女性  ペア  午後  369.0    20
                ID   年代   性別   条件  時間帯         反応時間         正答数
count   120.000000  120  120  120  120   118.000000  120.000000
unique         NaN    3    2    2    2          NaN         NaN
top            NaN  60代   女性   ペア   午前          NaN         NaN
freq           NaN   44   63   60   60          NaN         NaN
mean     60.500000  NaN  NaN  NaN  NaN   391.940678   15.758333
std      34.785054  NaN  NaN  NaN  NaN   101.312776    2.376339
min       1.000000  NaN  NaN  NaN  NaN   210.000000    9.000000
25%      30.750000  NaN  NaN  NaN  NaN   339.000000   14.000000
50%      60.500000  NaN  NaN  NaN  NaN   377.000000   16.000000
75%      90.250000  NaN  NaN  NaN  NaN   422.500000   17.000000
max     120.000000  NaN  NaN  NaN  NaN  1050.000000   20.000000
```

8.3 進め方の目安（第6コマの手順）

1. **眺める**（15分）：一覧と describe、groupby で一通り眺める
2. **気づきを1つ選ぶ**（5分）：一文で言える形にする
3. **確かめる**（15分）：集計で数字の裏づけを取る。比べる場合は分布の格差を見る
4. **グラフに仕上げる**（25分）：タイトル＝主張、軸は0から、単位と件数
5. **見直す**（10分）：下のチェックリストで自己確認して提出

8.4 提出前チェックリスト

□ 気づきは一文で書いたか
□ グラフのタイトルが、主張になっているか
□ 比べる相手が同じ図に入っているか
□ 軸が0から始まっているか（量のグラフの場合）
□ 単位と件数（n＝）を書いたか
□ プログラムが、もう一度実行しても同じ図になるか

8.5 振り返り

- お互いのグラフを見て、「一目で主張が伝わるか」を確かめ合います
- 「平均だけでは分からない姿を、グラフで知る」——これがこの講座の芯でした（第1コマ）

用語のミニ辞典

- **Thonny**：Python本体と編集画面が一緒になった初心者向けソフト。F5キーで実行する
- **変数（統計）**：測ったり記録したりする項目。尺度（名義・順序・間隔・比）がある
- **DataFrame**：pandasの表全体。Excelのシートのようなもの
- **集計**：データを種類ごとに分けて合計・平均などを出すこと
- **分布**：値のばらつき具合。ヒストグラムや箱ひげ図で見る
- **仮説**：「もし○○なら、△△になるはず」という、確かめられる形の予想
- **母集団と標本**：本当に知りたい全体が母集団、実際に集めたデータが標本（サンプル）
- **混同行列**：推測の正誤の数を数える表。行と列が同じカテゴリで、斜めが正解
- **度量衡**：長さ（度）・かさ（量）・重さ（衡）を測る、昔からの共通のものさし・単位の体系
- **分割表**：クロス集計表の別名。2つの変数の組み合わせで人数などを数える

困ったときの対処表

症状	対処
真っ赤なエラーが出た	一番下の行を読む。かっこ・スペル・全角文字を確認
グラフが表示されない	`plt.show()` の書き忘れ。保存してから、もう一度実行（F5）
日本語が四角になる	matplotlib-fontja を使う（第5コマ）。**Quarto のレンダリングは Thonny とは別の Python が使われる**ことがあるので、そのPython にも入っているか確認する。教材のコードは、matplotlib-fontja が使えない場合に入っている日本語フォント（IPAexゴシック／Noto Sans CJK／游ゴシック等）を自動で探す
変数が無いと言われる	プログラムは上から順に実行される。必要な行の位置を確認
CSVが文字化け・見つからない	`encoding` の指定／data フォルダが隣にあるか確認
保存した図が見当たらない	`savefig` は `show` の前。同じフォルダを探す

A Slide Index

A.1 1日目

1. データ可視化(導入編)
2. Python入門
3. データ集め、データ(変数)の尺度
4. 表のいろいろ・表作成

A.2 2日目

5. グラフのいろいろ・伝わるデザイン・グラフ作成
6. データを眺めて、気づいたことをグラフだけで納得させる(データ分析)
7. 仮説を納得させるグラフを作るためのデータ集め
8. テスト「データを眺めて、気づいたことをグラフで納得させよ」